

Fișă de lucru

Șirul lui Fibonacci în natură

1. Teorie

Șirul lui Fibonacci este un șir de numere în care fiecare termen se obține prin adunarea celor doi termeni anteriori.

Definiția matematică poate fi scrisă sub formă de funcție definită pe ramuri:

$$F(n) = \begin{cases} 0, & n = 0 \\ 1, & n = 1 \\ F(n-1) + F(n-2), & n \geq 2 \end{cases}$$

Primii termeni ai șirului sunt:

0, 1, 1, 2, 3, 5, 8, 13, 21, 34,...

Observăm:

$$1 = 0 + 1$$

$$5 = 3 + 2$$

$$2 = 1 + 1$$

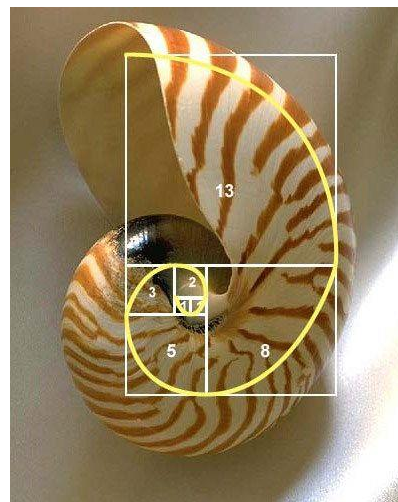
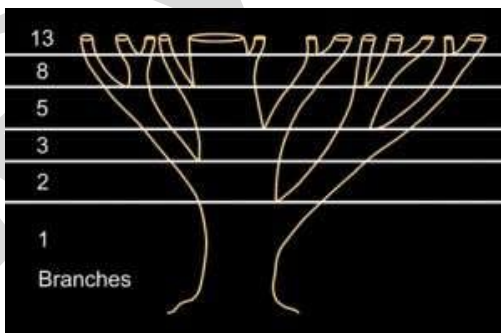
$$8 = 5 + 3$$

$$3 = 2 + 1$$

$$13 = 8 + 5$$

Exemple pentru șirul lui Fibonacci în natură:

- Numărul petalelor unei flori
- Spiralele semințelor de floarea-soarelui
- Forma cochiliilor melcilor
- Ramificarea copacilor
- Dispunerea frunzelor pe tulpină





2. Reprezentarea algoritmului

Pentru a calcula termenii șirului:

- Pornim de la primele două elemente
- Calculăm termenul nou prin adunare
- Repetăm procesul

3. Exemple rezolvate

• **Exemplul 1:** Se consideră șirul lui Fibonacci, definit prin relația că fiecare termen se obține ca sumă a celor doi termeni anteriori. Să se afișeze pe ecran primii 10 termeni ai șirului Fibonacci, în ordinea generării lor.

Pasul 1 – inițializarea variabilelor

```
a = 0  
b = 1
```

a – termenul curent
b – termenul următor

Pasul 2 – repetarea calculului

Vom folosi o structură repetitivă *for*.

```
for i in range(10):
```

Pasul 3 – afișarea și actualizarea valorilor

```
print(a)  
c = a + b  
a = b  
b = c
```

Programul complet:

```

a = 0
b = 1
for i in range(10):
    print(a)
    c = a + b
    a = b
    b = c

```

Tabel de test

Pas	a	b	c	Valoare afișată
Inițial	0	1	-	-
1	0	1	1	0
2	1	1	2	1
3	1	2	3	1
4	2	3	5	2
5	3	5	8	3
6	5	8	13	5
7	8	13	21	8
8	13	21	34	13
9	21	34	55	21
10	34	55	89	34

Algoritmul pornește de la primele două valori ale șirului Fibonacci, 0 și 1, memorate în variabilele a și b. La fiecare repetare a buclei se afișează termenul curent, apoi se calculează un nou termen prin adunarea valorilor a și b. Variabila c păstrează temporar această sumă. După calcul, valorile se actualizează: a primește valoarea lui b, iar b primește valoarea lui c. Astfel, programul „avansează” permanent în șir și generează succesiv termenii Fibonacci.

- **Exemplul 2:** Se citește de la tastatură un număr natural n, reprezentând poziția unui termen în șirul Fibonacci. Considerând că primul termen se află pe poziția 0, să se determine și să se afișeze valoarea termenului aflat pe poziția n.

Pasul 1 – analiza cazului particular

Pentru pozițiile mici ale șirului Fibonacci nu folosim regula generală de calcul:

- dacă $n = 0$, termenul este 0

- dacă $n = 1$, termenul este 1

Abia pentru $n \geq 2$ putem calcula termenii folosind suma celor doi termeni anteriori.

Pasul 2 – citirea valorii n

```
n = int(input(' n = '))
```

`int()` – asigură că valoarea citită este un număr întreg

Pasul 3 – verificarea cazului particular

```
if n == 0:  
    print('Termenul este: 0')  
elif n == 1:  
    print('Termenul este: 1')
```

Pasul 4 – inițializarea

```
a = 0  
b = 1
```

Pasul 5 – calculul repetitiv

For-ul este executat de la poziția 2 până la poziția $n + 1$ pentru că primele două cazuri ale șirului au fost tratate deja, iar funcția `range()` nu include capătul superior.

```
for i in range(2, n + 1):  
    c = a + b  
    a = b  
    b = c
```

Pasul 6 – afișarea rezultatului

```
print('Termenul este: ', b)
```

Programul complet:

```
n = int(input(' n = '))
if n == 0:
    print(' Termenul este: 0')
elif n == 1:
    print(' Termenul este: 1')
else:
    a = 0
    b = 1
    for i in range(2, n + 1):
        c = a + b
        a = b
        b = c
    print(' Termenul este: ', b)
```

Tabel de test: n = 7

Repetare	a(după actualizare)	b(după actualizare)	c
Inițial	0	1	-
2	1	1	1
3	1	2	2
4	2	3	3
5	3	5	5
6	5	8	8
7	8	13	13

Rezultat:

Termenul este: 13

Programul verifică mai întâi cazurile particulare ale șirului Fibonacci. Dacă n este 0 sau 1, rezultatul este cunoscut direct și nu mai este necesar niciun calcul. Pentru valorile mai mari sau egale cu 2, programul folosește regula generală a șirului Fibonacci și calculează succesiv termenii prin adunarea ultimelor două valori. Variabilele a și b memorează permanent ultimii doi termeni ai șirului.

4. Probleme propuse

Problema 1: Se citește de la tastatură un număr natural n . Să se afișeze primii n termeni ai șirului Fibonacci, fiecare pe aceeași linie, separați prin spațiu.

Problema 2: Se citește de la tastatură un număr natural n . Să se determine și să se afișeze termenul Fibonacci aflat pe poziția n (considerând că primul termen se află pe poziția 0).

Problema 3: Se citește de la tastatură un număr natural n . Să se calculeze și să se afișeze suma primilor n termeni ai șirului Fibonacci.

Problema 4: Să se genereze primii 20 de termeni ai șirului Fibonacci și să se afișeze doar termenii pari.

Problema 5(Cercetare în natură): Realizați un mic proiect în care să identificați cel puțin 3 exemple din natură unde apare șirul Fibonacci. Prezentați: imagini, explicații, legătura cu șirul Fibonacci.(Sugestii: floarea-soarelui, ananas, con de brad, cochilie)